Data structures and Algorithms

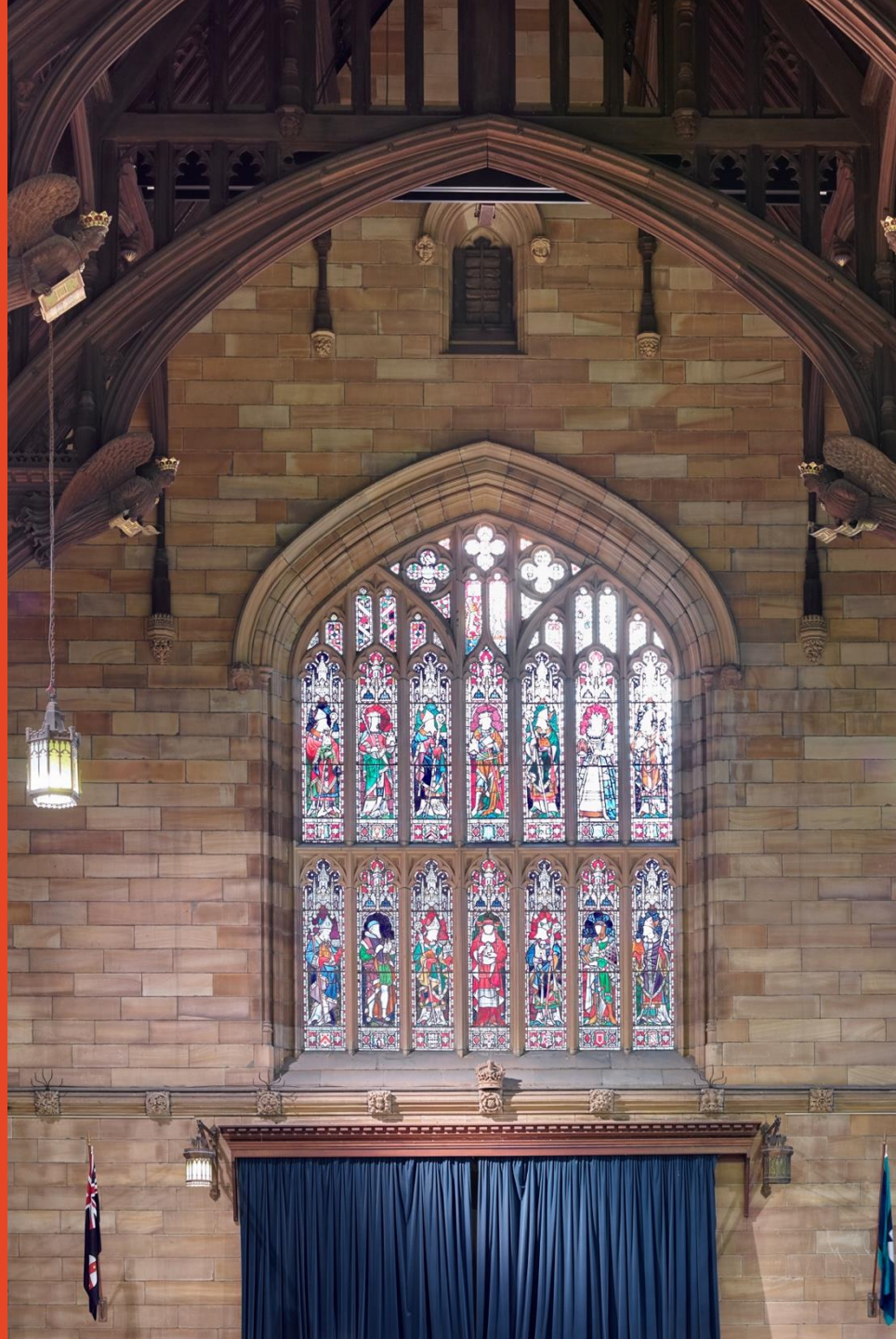
Lecture 6: Priority Queues [GT-5]

Version 1.0

Dr. Armin Chitizadeh
School of Computer Science

Inspired by lectures by Dr. Karlos Ishac and Dr. Andre van Renssen.

THE UNIVERSITY OF SYDNEY



Recap

- ADT Python Examples*
- Recommended Resources
- BST
- AVL

© Goodrich, Tamassia, Goldwasser

Menti

https://www.menti.com/alnyi42k28pc

Recommended Resources

The Teaching Team is putting together a list of additional resources for this course and will soon post this on Ed. For now here are some links to check out.

- Computerphile Youtube: https://www.youtube.com/watch?v=GazC3A4OQTE
- Abdul Bari's videos on later topics: https://www.youtube.com/playlist?list=PLDN4rrl48XKpZkf03iYFl-O29szjTrs_O
- Grock Learning Python for Beginners: https://groklearning.com/course/python-for-beginners/
- Python Exercises on HackerRank: https://www.hackerrank.com/domains/python

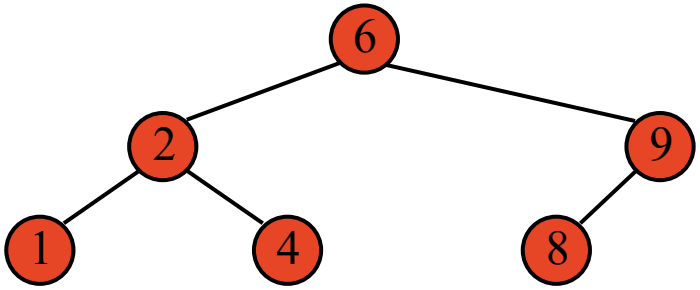
Binary Search Trees

A binary search tree is a binary tree storing keys (or key-value pairs) satisfying the following BST property

For any node v in the tree and
any node u in the left subtree of v and
any node w in the right subtree of v,

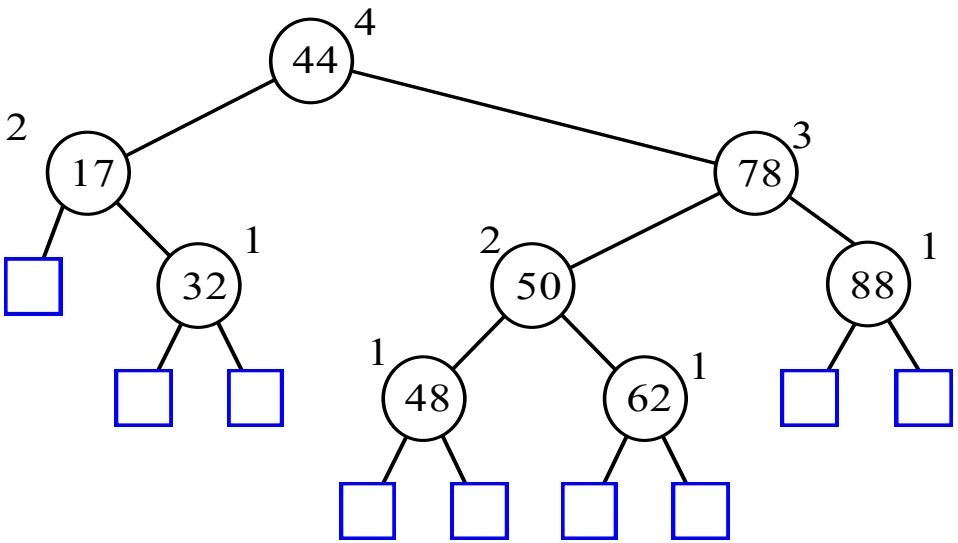
key(u) < key(v) < key(w)

Note that an inorder traversal of a binary search tree visits the keys in increasing order.



AVL Trees

AVL trees are rank-balanced trees, where r(v) is its height of the subtree rooted at v



Balance constraint: The ranks of the two children of every internal node differ by at most 1.

Fact: The height of an AVL tree storing n keys is O(log n).

Maps

The Map ADT

- get(k): if the map M has an entry with key k, return its associated value
- put(k, v): if key k is not in M, then insert (k, v) into the map M; else, replace the existing value associated to k with v
- remove(k): if the map M has an entry with key k, remove it
- size(), isEmpty()
- entrySet(): return an iterable collection of the entries in M
- keySet(): return an iterable collection of the keys in M
- values(): return an iterable collection of the values in M

© Goodrich, Tamassia, Goldwasser

Example

Operation	*Output*	*Map*
isEmpty()	**true**	Ø
put(5,*A*)	**null**	(5,*A*)
put(7,*B*)	**null**	(5,*A*),(7,*B*)
put(2,C)	**null**	(5,*A*),(7,*B*),(2,C)
put(8,*D*)	**null**	(5,*A*),(7,*B*),(2,C),(8,*D*)
put(2,*E*)	C	(5,*A*),(7,*B*),(2,*E*),(8,*D*)
get(7)	*B*	(5,*A*),(7,*B*),(2,*E*),(8,*D*)
get(4)	**null**	(5,*A*),(7,*B*),(2,*E*),(8,*D*)
get(2)	*E*	(5,*A*),(7,*B*),(2,*E*),(8,*D*)
size()	4	(5,*A*),(7,*B*),(2,*E*),(8,*D*)
remove(5)	*A*	(7,*B*),(2,*E*),(8,*D*)
remove(2)	*E*	(7,*B*),(8,*D*)
get(2)	**null**	(7,*B*),(8,*D*)
isEmpty()	**false**	(7,*B*),(8,*D*)

© Goodrich, Tamassia, Goldwasser

Sorted map ADT (extra methods)

firstEntry() returns the entry with smallest key; if map is empty, returns null

lastEntry() returns the entry with largest key; if map is empty, returns null

ceilingEntry(k) returns the entry with least key that is greater than or equal to k (or null, if no such entry exists)

floorEntry(k) returns the entry with greatest key that is less than or equal to k (or null, if no such entry exists)

lowerEntry(k) returns the entry with greatest key that is strictly less than k (or null, if no such entry exists)

higherEntry(k) returns the entry with least key that is strictly greater than k (or null, if no such entry exists)

subMap(k1,k2) returns an iteration of all the entries with key greater than or equal to k1 and strictly less than k2

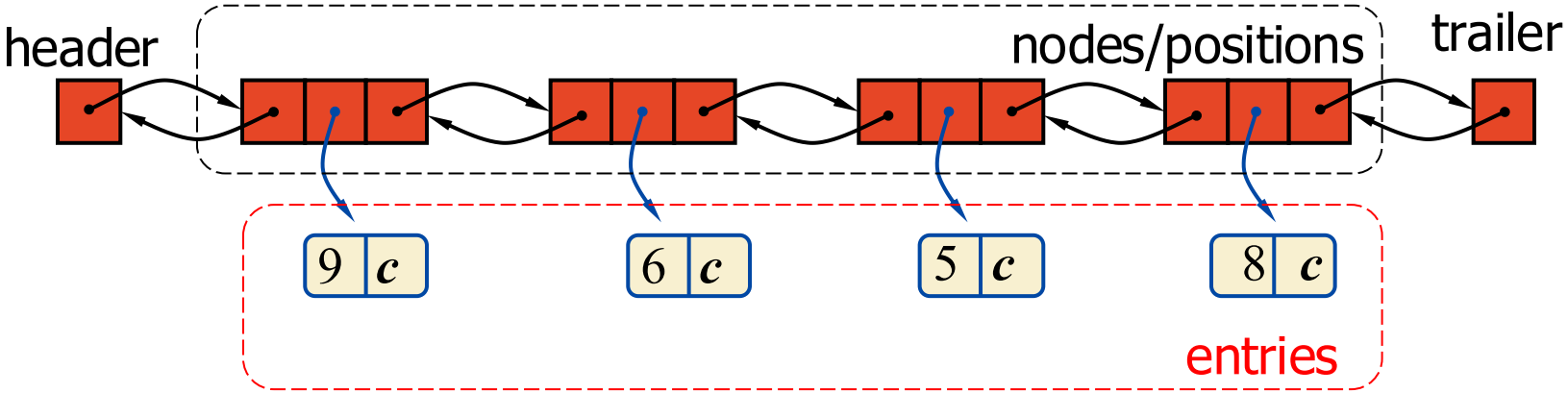
© Goodrich, Tamassia, Goldwasser

List-Based (unsorted) Map

We can implement a map using an unsorted list of key-item pairs

To do a get and put we may have to traverse the whole list, so those operations take O(n) time.

Only feasible if map is very small or if we put things at the end and do not need to perform many gets (i.e., system log)



Tree-Based (sorted) Map

We can implement a sorted map using an AVL tree, where each node stores a key-item pair

To do a get or a put we search for the key in the tree, so these operations take O(h) time, which can be O(log n) if the tree is balanced.

Only feasible if there is a total ordering on the keys.

Priority Queue ADT

Special type of ADT map to store a collection of key-value items where we can only remove smallest key:

- insert(k, v): insert item with key k and value v
- remove_min(): remove and return the item with smallest key
- min(): return item with smallest key
- size(): return how many items are stored
- is_empty(): test if queue is empty

We can also have a max version of this min version, but we cannot use both versions at once.

Interactive Example

A sequence of priority queue methods:

Method	Return value	Priority queue
insert(5,A)		{(5,A)}
insert(9,C)		{(5,A),(9,C)}
insert(3,B)		{(3,B),(5,A),(9,C)}
min()	(3,B)	{(3,B),(5,A),(9,C)}
remove_min()	(3,B)	{(5,A),(9,C)}
insert(7,D)		{(5,A),(7,D),(9,C)}
remove_min()	(5,A)	{(7,D),(9,C)}
remove_min()	(7,D)	{(9,C)}
remove_min()	(9,C)	{}
is_empty()	true	{}

SPEED CHALLENGE 6A

Write down the final priority queue after performing these steps.

Method

insert(3,3)

insert(6,7)

min()

insert(2,9)

insert(9,2)

insert(7,6)

remove_min()

insert(2,2)

Insert(4,2)

remove_min()

is_empty()

remove_min()

P = (lecturer will demonstrate in Python)

Application: Stock Matching Engines

At the heart of modern stock trading systems are highly reliable systems known as **matching engines,** which match the stock trades of buyers and sellers.

Buyers post bids to buy a number of shares of a given stock at or below a specified price

Sellers post offers (asks) to sell a number of shares of a given stock at or above a specified price.

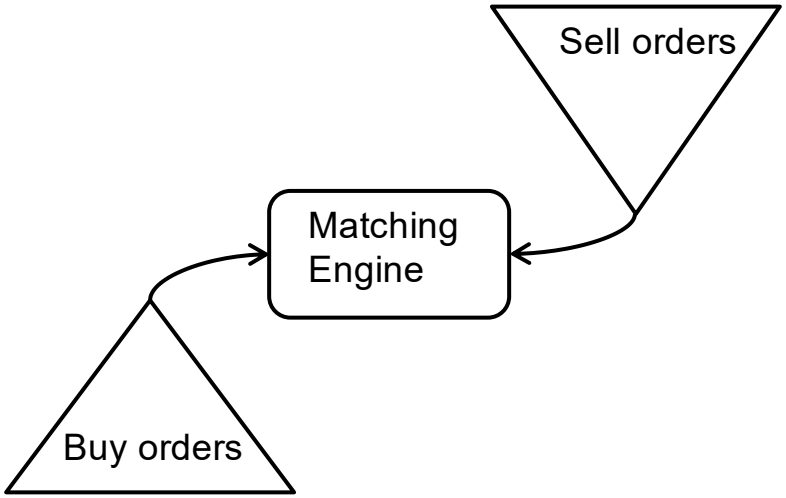
STOCK: EXAMPLE.COM

Buy Orders			Sell Orders		
Shares	Price	Time	Shares	Price	Time
1000	4.05	20 s	500	4.06	13 s
100	4.05	6 s	2000	4.07	46 s
2100	4.03	20 s	400	4.07	22 s
1000	4.02	3 s	3000	4.10	54 s
2500	4.01	81 s	500	4.12	2 s
			3000	4.20	58 s
			800	4.25	33 s
			100	4.50	92 s

Application: Stock Matching Engines

Buy and sell orders are organized according to a price-time priority, where price has highest priority and time is used to break ties

When a new order is entered, the matching engine determines if a trade can be immediately executed and if so, then it performs the appropriate matches according to price-time priority.



STOCK: EXAMPLE.COM

Buy Orders			Sell Orders		
Shares	Price	Time	Shares	Price	Time
1000	4.05	20 s	500	4.06	13 s
100	4.05	6 s	2000	4.07	46 s
2100	4.03	20 s	400	4.07	22 s
1000	4.02	3 s	3000	4.10	54 s
2500	4.01	81 s	500	4.12	2 s
			3000	4.20	58 s
			800	4.25	33 s
			100	4.50	92 s

Application: Stock Matching Engines

A matching engine can be implemented with two **priority queues**, one for buy orders and one for sell orders.

This data structure performs element removals based on priorities assigned to elements when they are inserted.

```
while True:
    bid ← buy_orders.remove_max()
    ask ← sell_orders.remove_min()
    if bid.price ≥ ask.price then
        carry out trade (bid, ask)
    else
        buy_orders.insert(bid)
        sell_orders.insert(ask)
```

STOCK: EXAMPLE.COM

Buy Orders			Sell Orders		
Shares	Price	Time	Shares	Price	Time
1000	4.05	20 s	500	4.06	13 s
100	4.05	6 s	2000	4.07	46 s
2100	4.03	20 s	400	4.07	22 s
1000	4.02	3 s	3000	4.10	54 s
2500	4.01	81 s	500	4.12	2 s
			3000	4.20	58 s
			800	4.25	33 s
			100	4.50	92 s

Two groups of people
How to place and find an item

Photo by Annie Spratt on Unsplash

Photo by Wonderlane on Unsplash

Sequence-based Priority Queue

Unsorted list implementation

- insert in O(1) time since we can insert the item at the beginning or end of the sequence
- remove_min and min in O(*n*) time since we have to traverse the entire list to find the smallest key

Sorted list implementation

- insert in O(n) time since we have to find the place where to insert the item
- remove_min and min in O(1) time since the smallest key is at the beginning

Method	Unsorted List	Sorted List
size, isEmpty	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$
min, removeMin	$O(n)$	$O(1)$

Priority Queue Sorting

We can use a priority queue to sort a list of keys:

1. iteratively insert keys into an empty priority queue
2. iteratively remove_min to get the keys in sorted order

Complexity analysis:

- n insert operations
- n remove_min operations

Either sequence-based implementation take $O(n^2)$

```
def priority_queue_sorting(A):
    pq ← new priority queue
    n ← size(A)
    for i in [0:n] do
        pq.insert(A[i])
    for i in [0:n] do
        A[i] = pq.remove_min()
```

Method	Unsorted List	Sorted List
size, isEmpty	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$
min, removeMin	$O(n)$	$O(1)$

Selection-Sort

Variant of pq-sort using unsorted sequence implementation:

1. inserting elements with n insert operations takes O(n) time
2. removing elements with n remove_min operations takes $O(n^2)$

Can be done in place
(no need for extra space)

Top level loop invariant:

- A[0:i] is sorted
- A[i:n] is the priority queue
  and all ⩾ A[i-1]

```
def selection_sort(A):
    n ← size(A)
    for i in [0:n] do
        # find s ⩾ i minimizing A[s]
        s ← i
        for j in [i:n] do
            if A[j] < A[s] then
                s ← j
        # swap A[i] and A[s]
        A[i], A[s] ← A[s], A[i]
```

Selection-Sort Example

i	A	s
0	7, 4, 8, 2, 5, 3, 9	3
1	2, 4, 8, 7, 5, 3, 9	5
2	2, 3, 8, 7, 5, 4, 9	5
3	2, 3, 4, 7, 5, 8, 9	4
4	2, 3, 4, 5, 7, 8, 9	4
5	2, 3, 4, 5, 7, 8, 9	5
6	2, 3, 4, 5, 7, 8, 9	6

```
def selection_sort(A):
    n ← size(A)
    for i in [0:n] do
        # find s ⩾ i minimizing A[s]
        s ← i
        for j in [i:n] do
            if A[j] < A[s] then
                s ← j
        # swap A[i] and A[s]
        A[i], A[s] ← A[s], A[i]
```

Selection-Sort

	8
	5
	2
	6
	9
	3
	1
	4
	0
	7

Insertion-Sort

Variant of pq-sort using sorted sequence implementation:

1. inserting elements with n insert operations takes $O(n^2)$ time
2. removing elements with n remove_min operations takes $O(n)$

Can be done in place
(no need for extra space)

Top level loop invariant:

- A[0:i] is the priority queue (and thus sorted)
- A[i:n] is yet-to-be-inserted

```
def insertion_sort(A):
    n ← size(A)
    for i in [1:n] do
        x ← A[i]
        # move forward entries > x
        j ← i
        while j > 0 and x < A[j-1] do
            A[j] ← A[j-1]
            j ← j - 1
        # if j>0 ⇒ x ≥ A[j-1]
        # if j<i ⇒ x < A[j+1]
        A[j] ← x
```

Insertion-Sort Example

i	A	j
1	7, 4, 8, 2, 5, 3, 9	0
2	4, 7, 8, 2, 5, 3, 9	2
3	4, 7, 8, 2, 5, 3, 9	0
4	2, 4, 7, 8, 5, 3, 9	2
5	2, 4, 5, 7, 8, 3, 9	1
6	2, 3, 4, 5, 7, 8, 9	6

```
def insertion_sort(A):
    n ← size(A)
    for i in [1:n] do
        x ← A[i]
        # move forward entries > x
        j ← i
        while j > 0 and x < A[j-1] do
            A[j] ← A[j-1]
            j ← j - 1
        # if j>0 ⇒ x ≥ A[j-1]
        # if j<i ⇒ x < A[j+1]
        A[j] ← x
```

Insertion-Sort

6 5 3 1 8 7 2 4

By Swfung8 - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=14961606

Challenge 6B

Consider the following image and answer the following questions:

1. Use selection sort to sort the books, write down each step.
2. Use insertion sort to sort the books, write down each step.
3. Which do you feel performed better?
4. How might you define and print the unsorted books in Python?

[5, 3, 14, 4, 2, 6, 1]

Challenge 6B

1. Use selection sort to sort the books, write down each step.
2. Use insertion sort to sort the books, write down each step.

Approximately 7-8 steps each

[1, 2, 3, 4, 5, 6, 14]

Which is better?

[1, 2, 3, 4, 5, 6, 14]

Challenge 6B

4. How might you define and print the unsorted books in Python?

```
unsorted_books = [5, 3, 14, 4, 2, 6, 1]

for book in unsorted_books:
    print(book)
```

```
unsorted_books = {
    5: "YYH5",
    3: "YYH3",
    14: "YYH14",
    4: "YYH4",
    2: "YYH2",
    6: "YYH6",
    1: "YYH1"
}
for key, value in unsorted_books.items():
    print(f"{key}: {value}")
```

Menti competition!

https://www.menti.com/alxu167w27r4

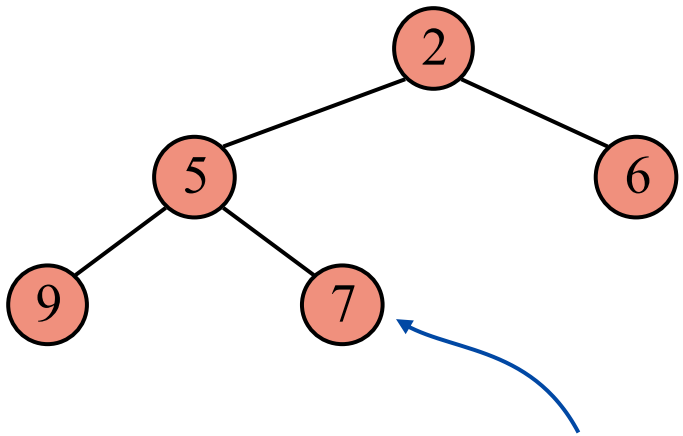
Break

Heap data structure (min-heap)

A heap is a binary tree storing (key, value) items at its nodes, satisfying the following properties:

1. Heap-Order: for every node m ≠ root,

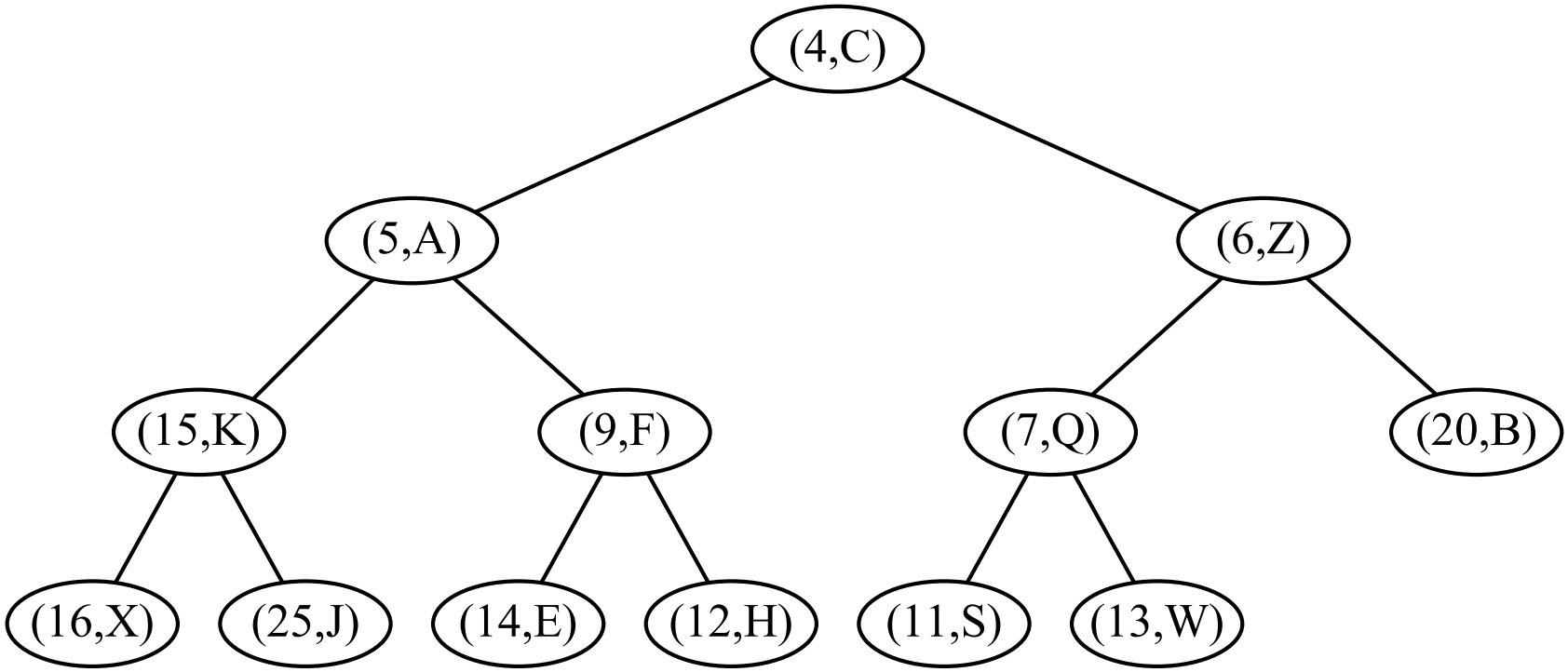
   key(m) ≥ key(parent(m))



2. Complete Binary Tree: let h be the height
   - every level $i < h$ is full (i.e., there are 2^i nodes)
   - remaining nodes take leftmost positions of level h

The last node is the rightmost node of maximum depth

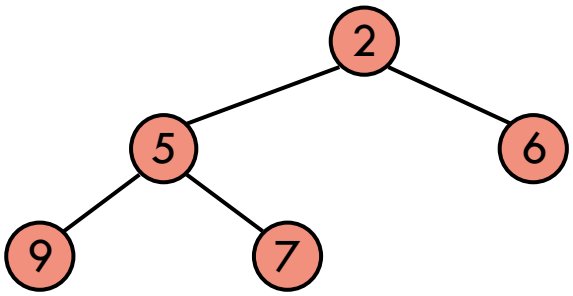
Example

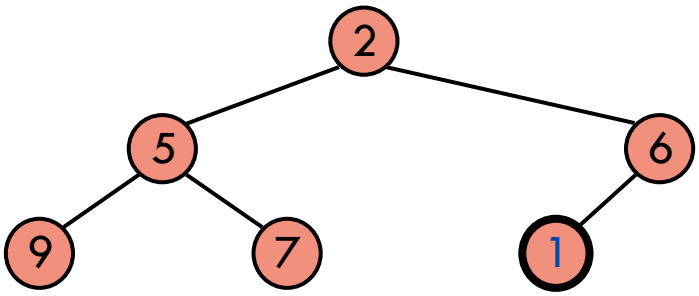


Insertion into a Heap

- Create a new node with given key
- Find location for new node
- Restore the heap-order property

insert(1)

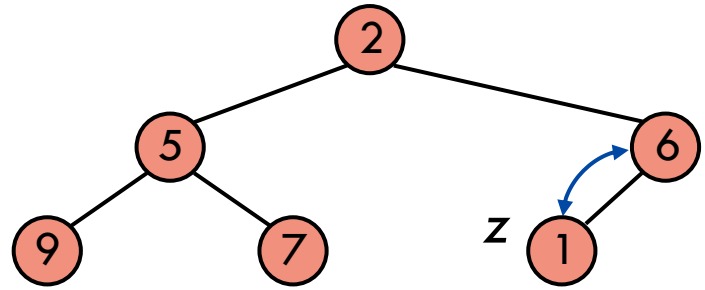


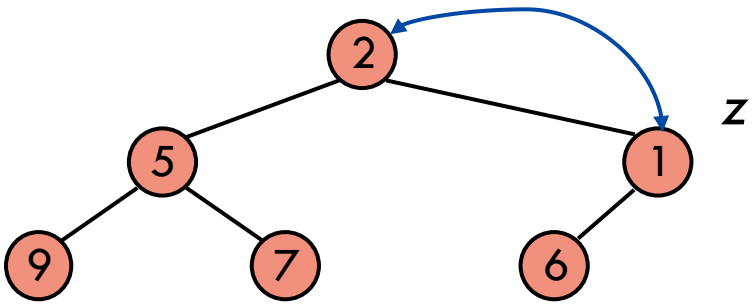


Upheap

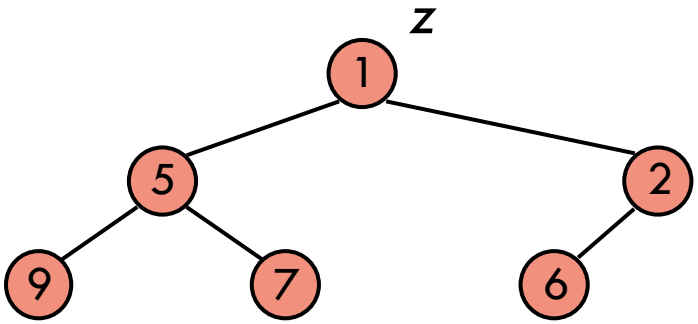
Restore heap-order property by swapping keys along upward path from insertion point

```
def up_heap(z):
    while z ≠ root and
    key(parent(z)) > key(z) do
        swap key of z and parent(z)
        z ← parent(z)
```





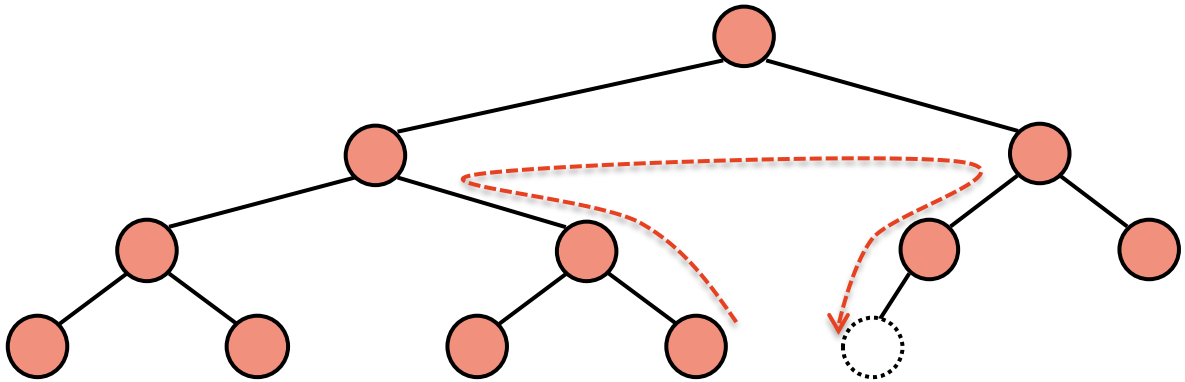
Complexity: O(log n) time because the height of the heap is log n



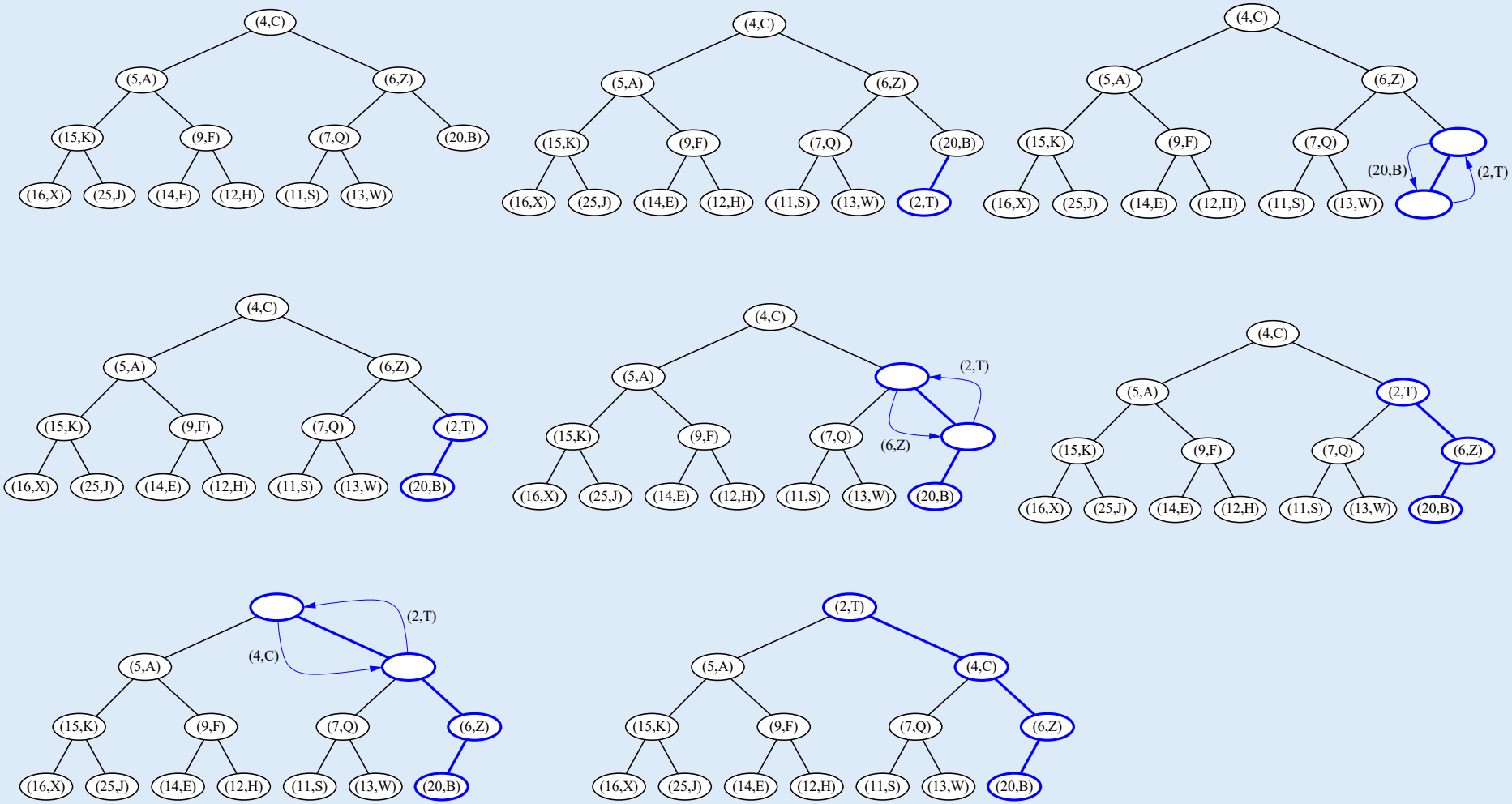
Finding the position for insertion

- start from the last node
- go up until a left child or the root is reached
- if we reach the root then need to open a new level
- otherwise, go to the sibling (right child of parent)
- go down left until a leaf is reached

Complexity of this search is O(log n) because the height is log n.
Thus, overall complexity of insertion is O(log n) time



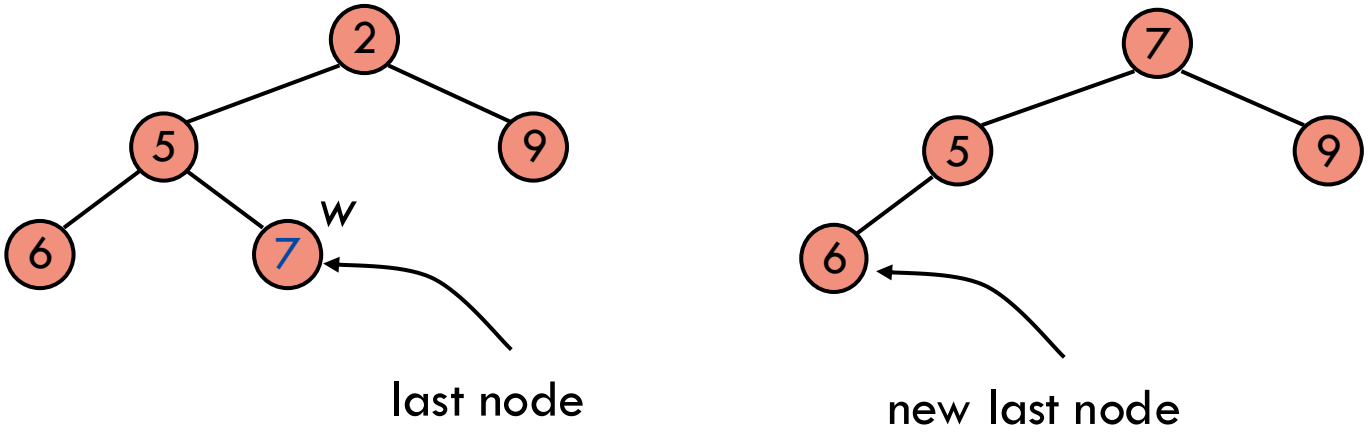
Example insertion



Removal from a Heap

- Replace the root key with the key of the last node *w*
- Delete *w*
- Restore the heap-order property

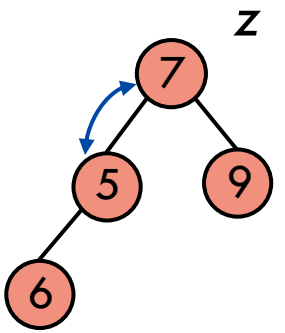
remove_min()

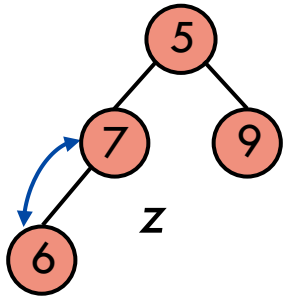


Downheap

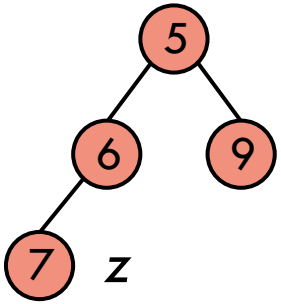
Restore heap-order property by swapping keys along downward path from the root

```
def down_heap(z):
    while z has child with
      key(child) < key(z) do
        x ← child of z with smallest key
        swap keys of x and z
        z ← x
```

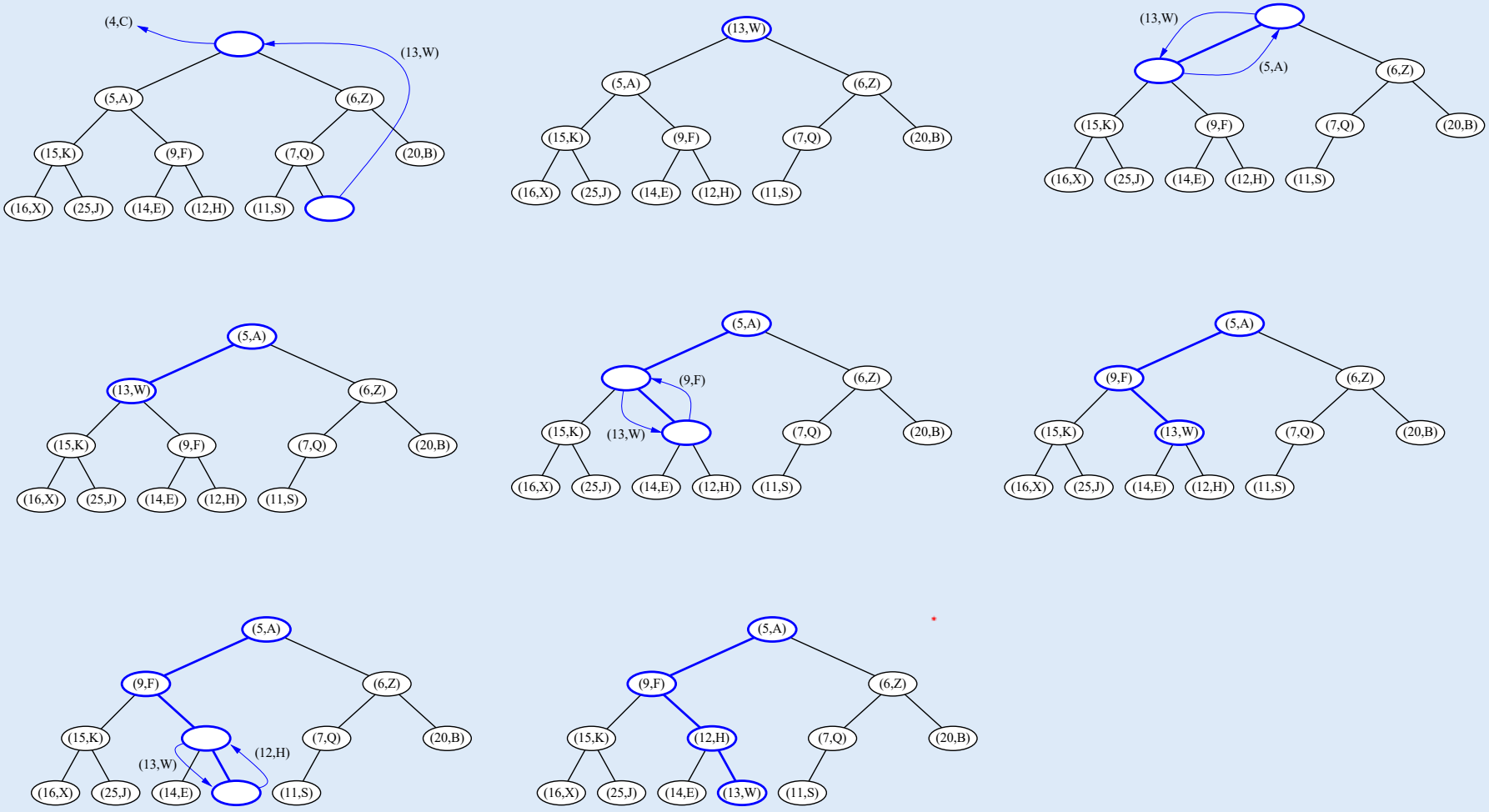




Complexity: O(log n) time because the height of the heap is log n



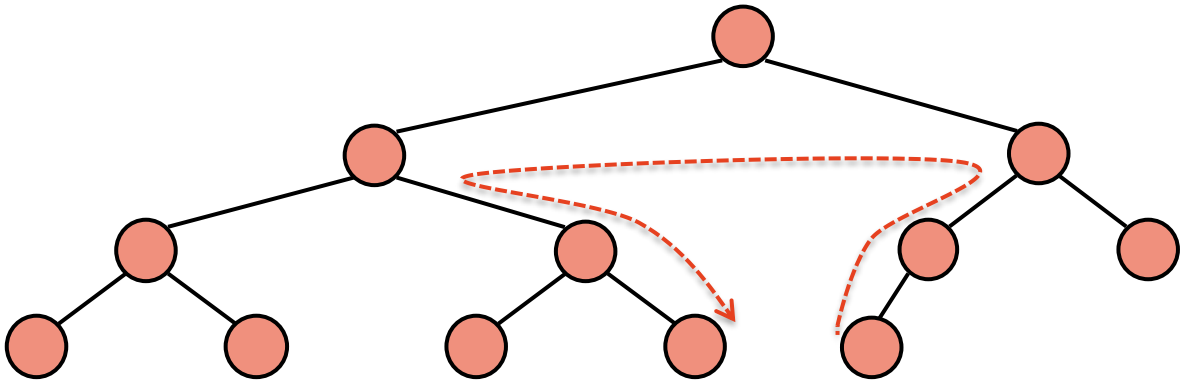
Example removal



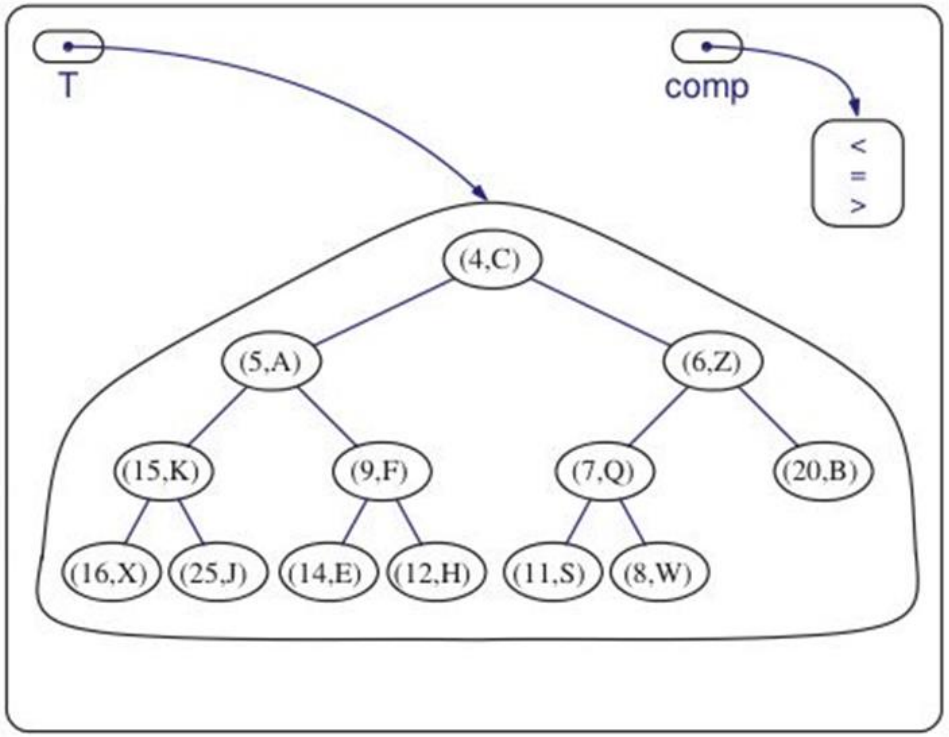
Finding next last node after deletion

- start from the (old) last node
- go up until a right child or the root is reached
- if we reach the root then need to close a level
- otherwise, go to the sibling (left child of parent)
- go down right until a leaf is reached

Complexity of this search is O(log n) because the height is log n.
Thus, overall complexity of deletion is O(log n) time



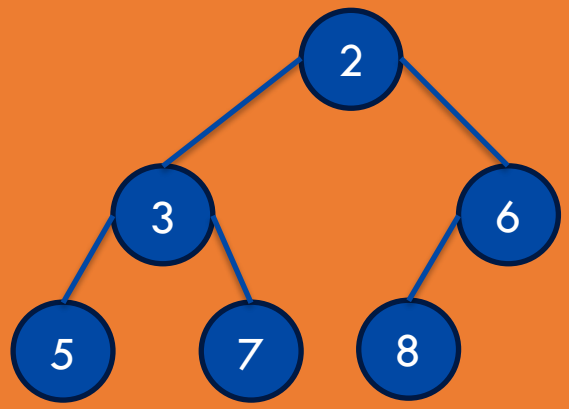
Heap-based implementation of a priority queue



Operation	Time
size, isEmpty	$O(1)$
min,	$O(1)$
insert	$O(\log n)$
removeMin	$O(\log n)$

Challenge 6C

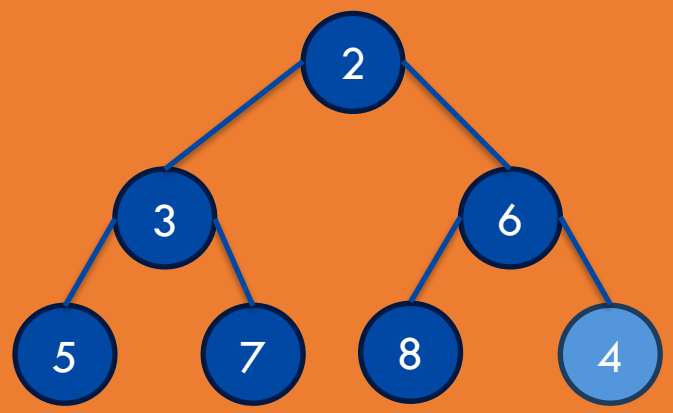
Restore the heap-order property based on the following actions

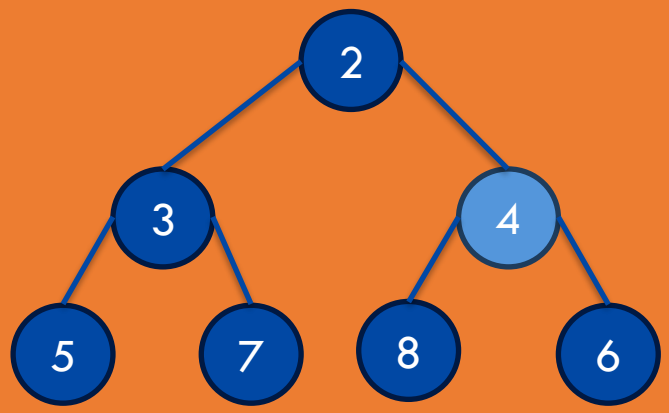


1. Insert the node 4 and restore the tree (show your answer)
2. Using your result from (1) remove the last node and restore the tree

Challenge 6C

Restore the heap-order property based on the following actions

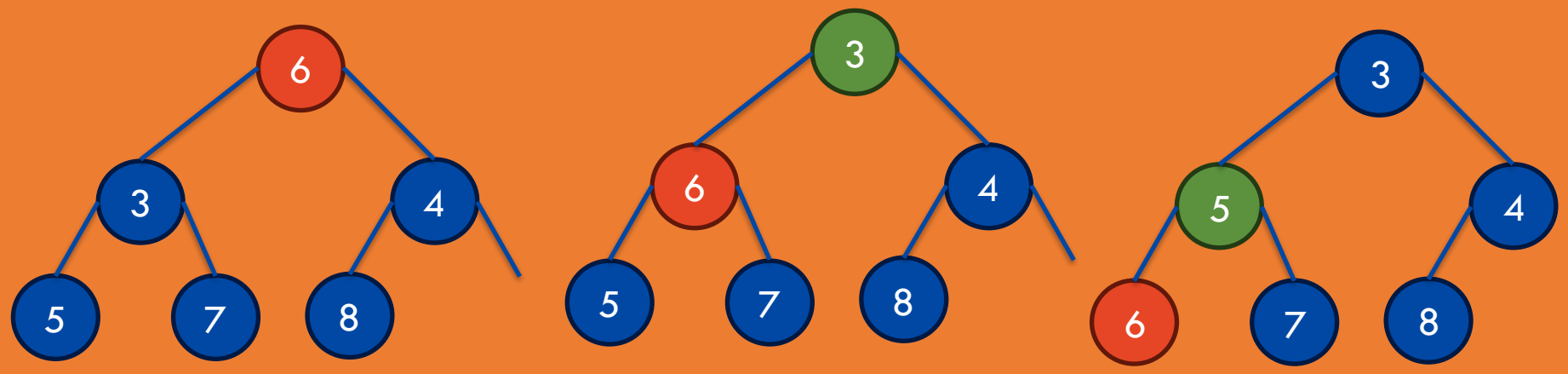




1. Insert the node 4 and restore the tree (show your answer)
2. Using your result from (1) remove the last node and restore the tree

Challenge 6C

Restore the heap-order property based on the following actions



2. Using your result from (1) remove the last node and restore the tree

Heap-Sort

Consider a priority queue with *n* items implemented with a heap:

- the space used is O(*n*)
- methods insert and remove_min take O(log n)

Recall that priority-queue sorting uses:

- n insert ops
- n remove_min ops

Heap-sort is the version of priority-queue sorting that implements the priority queue with a heap. It runs in O(n log n) time.

Heap-in-array implementation

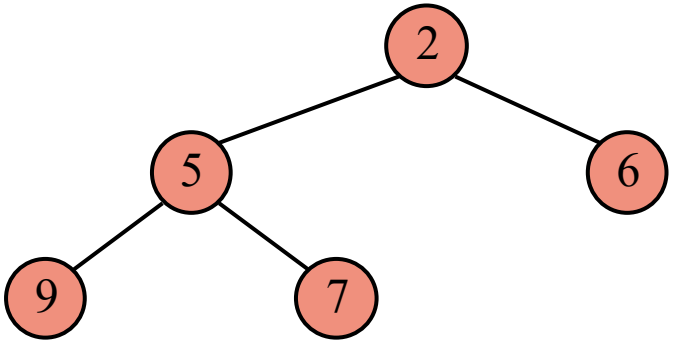
We can represent a heap with *n* keys by means of an array of length *n*

Special nodes:

- root is at 0
- last node is at n-1

For the node at index i:

- the left child is at index $2i+1$
- the right child is at index $2i+2$
- Parent is at index $\lfloor (i-1)/2 \rfloor$



2	5	6	9	7
0	1	2	3	4

Refinements and Generalization

Heap-sort can be arranged to work in place using part of the array for the output and part for the priority queue

A heap on n keys can be constructed in O(n) time. But the n remove_min still take O(n log n) time

Sometimes it is useful to support a few more operations (all are given a pointer to e):

- remove(e): Remove item e from the priority queue
- replace_key(e, k): update key of item e with k
- replace_value(e, v): update value of item e with v

Summary: Priority queue implementations

Method	Unsorted List	Sorted List	Heap
size, isEmpty	$O(1)$	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$	$O(\log n)$
min	$O(n)$	$O(1)$	$O(1)$
removeMin	$O(n)$	$O(1)$	$O(\log n)$
remove	$O(1)$	$O(1)$	$O(\log n)$
replaceKey	$O(1)$	$O(n)$	$O(\log n)$
replaceValue	$O(1)$	$O(1)$	$O(1)$

Implementing a Priority Queue

Entries: An object that keeps track of the associations between keys and values

Comparators: A function or an interface to compare entry objects

compare(a, b): returns an integer i such that

- $i < 0$ if $a \prec b$,
- $i = 0$ if $a = b$
- $i > 0$ if $a \succ b$

Warning: do not assume that compare(a,b) is always -1, 0, 1

Discovery Activity

You are a Doctor at the Hospital Emergency Room and are about to end your shift when several patients that have been involved in a car accident arrive at the Hospital. Given the following patients and their conditions, answer the following questions.

Patient	Age	Injury
Alice	30	Fractured Foot
Bob	55	Stroke
Charlie	22	Internal Bleeding
David	40	Severe Burns
Eve	70	Boken Arm
Fernando	24	Broken Leg

Injury	Priority Level
Life-threatening (stroke)	1
Critical (Severe Burns)	2
Serious (Internal Bleeding)	3
Moderate (Broken Limb)	4
Mild (Fractured Bone)	5
Minor (Bruises)	6-10

1. Build a priority list based on Injury Severity prioritizing older age patients if the severity level is the same.
2. Write simple Python or pseudocode for the PQ and printing the order of patients for the Doctor to call out.

Discovery Activity

1. Build a priority list based on Injury Severity prioritizing older age patients if the severity level is the same.

Patient	Age	Injury
Alice	30	Fractured Foot
Bob	55	Stroke
Charlie	22	Internal Bleeding
David	40	Severe Burns
Eve	70	Boken Arm
Fernando	24	Broken Leg

Injury	Priority Level
Life-threatening (stroke)	1
Critical (Severe Burns)	2
Serious (Internal Bleeding)	3
Moderate (Broken Limb)	4
Mild (Fractured Bone)	5
Minor (Bruises)	6-10

PQ = { Bob, David, Charlie, Eve, Fernando, Alice }

Discovery Activity

2. Write simple Python or pseudocode for the PQ and printing the order of patients for the Doctor to call out.

```
import heapq
```

```
patients = [
    {"name": "Alice", "age": 30, "injury": "Fractured Foot", "severity": 5},
    {"name": "Bob", "age": 55, "injury": "Stroke", "severity": 1},
   ....]
```

```
def patient_comparator(patient):
     return (patient["severity"], patient["age"] * -1)
```

```
pq = []
for patient in patients:
    heapq.heappush(pq, (patient_comparator(patient), patient))
```

Discovery Activity

2. Write simple Python or pseudocode for the PQ and printing the order of patients for the Doctor to call out.

```
print("Patient Treatment Order:")
while pq:
    _, patient = heapq.heappop(pq)
        print(f"{patient['name']} (Injury: {patient['injury']}, Severity: {patient['severity']},
        Age: {patient['age']})")
```

```
Treating Bob with Stroke (Severity: 1, Age: 55)
Treating David with Severe Burns (Severity: 2, Age: 40)
……
```

Fully anonymous feedback form for COMP9123 students – Fill out form